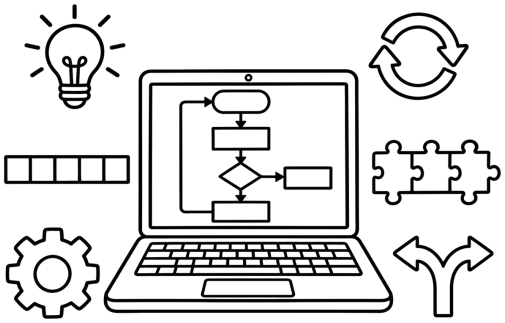
ICT

INFORMATION AND COMMUNICATION TECHNOLOGY

GRADE 11



UNIT 1 - PART 2
Programming

Student Workbook

"The King of ICT"

CHAMEEN RATHNASENA

Senior Lecturer in IT
Motivational Speaker
BSc (Sp) Computer Science - University of Peradeniya
PGDE - Open University of Sri Lanka
MIT (Reading) - University of Sabaragamuwa

STUDENT DETAILS

Name: _____

School: _____

Institution: _____

Contact: _____

BEFORE WE START

What This Part Is About

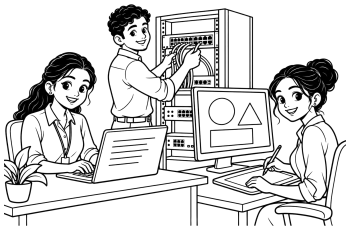
Unit 1 Programming - Part 2 of 2

AT THE END OF THIS PART YOU WILL BE ABLE TO

- [ ] Write Pascal programs with IF, IF ... ELSE, nested IF and CASE.
- [ ] Repeat statements with FOR ... DO, WHILE ... DO and REPEAT ... UNTIL.
- [ ] Put a control structure inside another one (nested structures).
- [ ] Declare arrays, store values in them and use them with loops.
- [ ] Divide a program into procedures and functions.
- [ ] Compare low-level and high-level languages and the paradigms.
- [ ] Explain the work of an assembler, an interpreter and a compiler.

HOW TO USE THIS BOOK

Read each chapter with your teacher, do the practical tasks and then write the exercises. The **WRITING SPACES** are for the notes your teacher gives you in class - write them neatly: they are your revision notes for the O/L examination.



THE WORDS YOU WILL LEARN

IF ... THEN ... ELSE	Nested IF	CASE	FOR ... DO
DOWNTO	WHILE ... DO	REPEAT ... UNTIL	Infinite loop
Nested structure	Array	Index	Element
Sub-program	Procedure	Function	Machine language
Assembly language	High-level language	Paradigm	Object
Class	Assembler	Interpreter	Compiler

"A good program chooses wisely, repeats tirelessly and remembers everything."

- THE KING OF ICT

WRITING SPACE 11 - Pascal

write what your teacher tells you

CHAPTER 4

Selection and Repetition

? LIBRARY OR PLAYGROUND?

"If it rains, I go to the library. If not, I play cricket!"

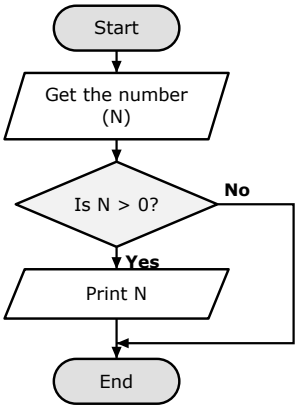
"And every day you repeat it until the term ends. That is selection AND repetition."

In Pascal, **IF** and **CASE** select what to do; **FOR**, **WHILE** and **REPEAT** repeat statements.



1.6 Selection: the IF statement

There are two types of IF statements. **i) IF ... THEN ... ENDIF**: the statement is executed only if the condition is satisfied. E.g. 1 - if the input number is positive, print it:



Flowchart

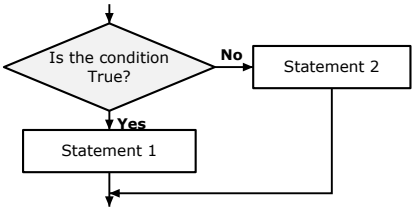
```
Begin
    Input N
    IF N > 0 THEN
        Print N
    ENDIF
End.
```

Pseudo code

```
1  Program positiveNo(input, output);
2  Var N : integer;
3  Begin
4      Writeln('Enter Number');
5      Read(N);
6      If N > 0 then
7          Writeln('Positive Number');
8  End.
```

Pascal code

ii) IF ... THEN ... ELSE ... ENDIF: if the condition is satisfied statement 1 is executed; if not, statement 2 is executed. E.g. 2 - the larger number from two unequal numbers (no semicolon before Else: the IF statement ends at Large := N2;).




```
IF Condition THEN
    Statement1
ELSE
    Statement2
ENDIF
```

```
1  program LargeNo(input,output);
2  Var N1,N2,Large: integer;
3  Begin
4      Writeln('Enter Two Numbers');
5      Read(N1,N2);
6      If N1 > N2 then
7          Large := N1
8      Else
9          Large := N2;
10     Writeln('Large Number is ', Large);
11 End.
```

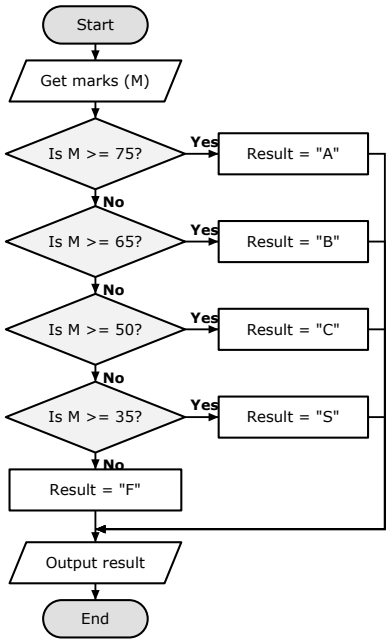
"IF you practise THEN you improve ELSE you stay the same. Choose well!"

- THE KING OF ICT

WRITING SPACE 12 - IF statements write what your teacher tells you

Nested IF - multiple conditions for a single variable

When there are multiple conditions one after the other, **nested IF** is used. E.g. 3 - the grade for the mark of a subject:



Flowchart

```
program GradeForMarks (input,output);
Var
    M: integer;
    Grade: char;
Begin
    Writeln('Enter Marks');
    Read(M);
    If M >= 75 then
        Grade := 'A'
    Else
      If M >= 65 then
          Grade := 'B'
      Else
        If M >= 50 then
            Grade := 'C'
        Else
          If M >= 35 then
              Grade := 'S'
          Else
              Grade := 'F';
    Writeln('Grade = ', Grade);
End.
```

Pascal code

The CASE statement - a variable with multiple conditions

Rather than using IF ... THEN ... ELSE ... ENDIF repeatedly, it is easier to use a **CASE** statement when one variable has multiple conditions.

```
program FindGrade(input,output);
var
    Marks : integer;
    Grade: char;
Begin
    Writeln('Enter Marks');
    Read(Marks);
    Case Marks of
        0..34 : Grade := 'W';
        35..49 : Grade := 'S';
        50..64 : Grade := 'C';
        65..74 : Grade := 'B';
        75..100 : Grade := 'A';
    Else
        Writeln('Invalid Marks');
    End;
    if (Marks >= 0) AND (Marks <= 100) then
        Writeln('Grade is ', Grade);
End.
```

The program FindGrade

Case Marks of lists the ranges of Marks; the statement of the matching range is executed. **Else** runs when Marks is in no range. The CASE ends with **End;**.

Marks	Result
0 - 34	'W'
35 - 49	'S'
50 - 64	'C'
65 - 74	'B'
75 - 100	'A'
other	Invalid Marks

The branches of the CASE flowchart (page 28)

1.7 Repetition

When the number of repetitions is known in advance, **FOR - DO** is used: **FOR Variable := Value_1 TO Value_2 DO** (method 1) or **FOR Variable := Value_1 DOWNTO Value_2 DO** (method 2). The variable and the values are integers; the repetition starts with Value_1 and ends with Value_2.

Repetition structure	Starting value	Ending value	Number of repetitions
FOR X := 1 TO 5 DO	1	5	5
FOR X := 0 TO 4 DO	0	4	5
FOR X := 5 TO 10 DO	5	10	6
FOR X := 10 DOWNTO 5 DO	10	5	6
FOR X := 4 DOWNTO 0 DO	4	0	5

```
Program print10Nos(input,output);
Var count : integer;
Begin
    For count := 1 to 10 do
        Writeln(count);
End.
```

Printing the values from 1 to 10

```
Program printReverse (input,output);
Var count : integer;
Begin
    For count := 10 downto 1 do
        Writeln(count);
End.
```

Printing the values from 10 to 1

```
program total_avg (input,output);
var I,num,total : integer;
    avg: real;
Begin
    total := 0;
    for I := 1 to 10 do
    begin
        writeln('Enter Number');
        read(num);
        total := total+num;
    end;
    avg := total/I;
    writeln('Total is ', total);
    writeln('Average is ',avg);
end.
```

Finding the total and average of ten numbers

IMPORTANT

A **block of statements** in a loop is written between **begin** and **end;** and indented properly.

When the number of repetitions is **not known** in advance, **WHILE ... DO** or **REPEAT ... UNTIL** is used.

WHILE ... DO (E.g. 1: while number > 0 do)

- Conditions are checked at the **beginning** of the loop.
- The statements inside the loop are executed only if the condition is true; they are never executed if the condition is false.
- The condition becomes false at the end of the repetition. If it never becomes false, the loop is an **infinite loop**.

REPEAT ... UNTIL

- The condition is not checked at the beginning: it is checked **after the statements are executed once**.
- The loop goes on while the condition is false and stops when the condition becomes **true**.
- If the condition never becomes true, the loop is an infinite loop.

```
number := 1;
while number <= 10 do
  number := number + 1;
```

WHILE E.g. 2

number starts at 1 and 1 is added each time: the loop stops when number is 11.

```
count := 0;
Repeat
  writeln('Pascal');
  count := count + 1
Until count > 5;
```

REPEAT E.g. 1

count starts at 0; the loop stops when count is 6: **Pascal** is printed 6 times.

```
sum := 0;
repeat
  sum := sum + 5;
  writeln(sum);
until sum < 50;
```

REPEAT E.g. 2

After one run sum is 5 and sum < 50 is true, so the loop stops: only **5** is printed.

```
sum := 0;
repeat
  sum := sum + 5;
  writeln(sum);
until sum >= 50;
```

REPEAT E.g. 3

It runs till sum >= 50 is true: 10 times, printing **5, 10 ... 50**.

WRITING SPACE 13 - FOR, WHILE and REPEAT

write what your teacher tells you

DO IT - PRACTICAL 4 (TEXTBOOK PAGES 26 - 33)

Run the selection and repetition programs of 1.6 and 1.7 in Free Pascal or Lazarus. **Ask your teacher first.**

1	Type **positiveNo**. Compile and run it with 25, then with -4. When is Positive Number printed?
2	Type **GradeForMarks** and run it with 80, 68, 52, 40 and 20. Then type **FindGrade** (CASE) and run it with the same marks and with 120.
3	Type **print10Nos** and **printReverse**. Run them and compare the outputs.
4	Type **total_avg**. Enter ten numbers. Check the total with a calculator.
5	Put REPEAT E.g. 2 and E.g. 3 into a program (var sum : integer;). How many numbers does each one print?

A What is the output?

8 marks

Run the programs of 1.6 in your head. Write the last line of the output.

Program	Input	The last line of the output
positiveNo	25	
positiveNo	-4	
LargeNo	18 27	
GradeForMarks	68	
GradeForMarks	34	
FindGrade	42	
FindGrade	75	
FindGrade	120	

WRITING SPACE 14 - The CASE statement

write what your teacher tells you

B Trace the loops

6 marks

Write the output of each script (2 marks each).

Script 1

```
count := 0;
Repeat
    writeln('ICT');
    count := count + 1
Until count > 2;
```

OUTPUT

.....

.....

.....

.....

Script 2

```
sum := 0;
repeat
    sum := sum + 5;
    writeln(sum);
until sum >= 20;
```

OUTPUT

.....

.....

.....

.....

Script 3

```
n := 10;
while n > 0 do
begin
    writeln(n);
    n := n - 3;
end;
```

OUTPUT

.....

.....

.....

.....

C Complete the program

6 marks

The program reads five numbers and says even or odd. Fill the gaps.

```
1  program evenOdd(input,output);
2  var num, count : integer;
3  begin
4      for count := 1 (1) 5 do
5      (2)
6          writeln('Enter Number');
7          read(num);
8          if num (3) 2 = 0 (4)
9              writeln('Even number')
10         (5)
11             writeln('Odd number');
12     (6) ;
13 end.
```

1. .....
2. .....
3. .....
4. .....
5. .....
6. .....

D Find the errors

6 marks

The program has six errors. Write the line number and the correct statement.

```
1  program Large No(input,output);
2  Var N1,N2,Large: integer
3  Begin
4      Writeln('Enter Two Numbers');
5      Read(N1,N2);
6      If N1 > N2
7          Large = N1
8      Else
9          Large := N2;
10     Writeln("Large Number is ", Large);
11 End
```

Line	The correct statement

"Repeat until you understand. That is the best loop of all."

- THE KING OF ICT

CHAPTER 5

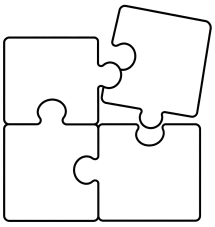
Nested Structures and Arrays

? THIRTY-FIVE MARKS, ONE NAME

"Our ICT teacher has the marks of 35 students. Does she need 35 variables?"

"No! One array keeps all of them under a single name."

Control structures can go inside each other (**nesting**), and an **array** stores many values of the same type under one name.



1.8 Nested control structures

Inside an algorithm, a control structure may be included inside another: a repetition inside another repetition, or a selection inside another selection. **1.8.1 Repetition inside selection** (left): an ascending or a descending sequence, by the user's choice. **1.8.2 Selection in repetition** (right): for ten numbers, odd or even, and the counts.

```
program orderNos(input,output);
var num:integer;
    cho:char;
begin
    writeln('Select Ascending(A) or Descending(D)');
    read(cho);
    if cho = 'A' then
    begin
        writeln('Ascending Order');
        for num := 1 to 6 do
            writeln(num);
    end;
    if cho = 'D' then
    begin
        writeln('Descending Order');
        for num := 6 downto 1 do
            writeln(num);
    end;
end.
```

orderNos: a FOR loop inside each IF

```
program rep_sel(input,output);
var num,rem,count,e_count,o_count:integer;
begin
    for count := 1 to 10 do
    begin
        writeln('Enter Number');
        read(num);
        rem := num mod 2;
        if rem = 0 then
        begin
            writeln('Even number ');
            e_count := e_count+1;
        end
        else
        begin
            writeln('Odd number ');
            o_count := o_count + 1;
        end;
    end;
    writeln(e_count,'Even Number/s');
    writeln(o_count ,'Odd Number/s');
end.
```

rep_sel: an IF ... ELSE inside the FOR loop

WRITING SPACE 15 - Nested control structures write what your teacher tells you

1.9 Arrays

Variables store data items in memory. Different variables with different names are needed for data items of the same type: five whole numbers need five variables.

```
Var p, q, r, s, t : integer;
    n1, n2, n3, n4, n5 : real;
```

1.9.1 Use of arrays. An **array** saves data items of the **same type** in memory using a **single variable identifier name**, without different variable names for each item.

1.9.2 Defining a one-dimensional array. An array uses a group of **adjoining memory spaces**:

```
Var Name_of_Array : array[first index .. last index] of data type
```

```
var marks : array [0..9] of integer;
```

The array marks can include 10 whole numbers: marks[0] to marks[9] (first index 0, array length 10).

1.9.3 Attributes of an array.

- The elements are positioned next to each other (adjacently).
- The index (sequential number) is written in **square brackets**: M[0].
- var M : array[0..4] of integer; has **5** elements, M[0] to M[4]. The first element has the lowest index, the last the highest.
- With var X : array[1..5] of integer; the indexes are X[1] to X[5].
- Only data items of the **same type** can be stored in an array.
- Any element can be accessed **randomly**, so a repetition can use the whole array.

```
1  var maths : array[0..39] of integer;
2      i,marks : integer;
3  for i := 0 to 39 do
4  begin
5      writeln('Enter marks');
6      read(marks);
7      maths[i] := marks;
8  end;
```

Entering the Maths marks of 40 students

WRITING SPACE 16 - Arrays — write what your teacher tells you

1.9.4 Assigning values to an array. An integer array with five elements:

```
1  var num : array[0..4] of integer;
2  num[0] := 45;
3  num[2] := 36; num[4] := 60;
4  num[1] := num[4] + 15;
5  num[3] := num[0] + num[2];
```

num[0]	num[1]	num[2]	num[3]	num[4]
45		**36**		**60**
45	**75**	**36**	**81**	**60**

"One name, many values: an array keeps your data neat."

- THE KING OF ICT

1.9.5 Printing the values of an array (num: 45, 75, 36, 81, 60):

Statement	What it prints
`writeln (num[3]);`	Print the 4th element (81)
`writeln (num[1], num[4]);`	Print the 2nd and 5th elements (75, 60)
`for x := 0 to 3 do writeln (num[x]);`	Print the first 4 elements (45, 75, 36, 81)
`for x := 2 to 4 do writeln (num[x]);`	Print the 3rd, 4th and 5th elements (36, 81, 60)
`for x := 0 to 4 do writeln (num[x]);`	Print all the elements (45, 75, 36, 81, 60)

E.g. - Entering the ICT marks of 35 students, finding the highest mark and the average.

```
1  program ictMarks(input,output);
2  var marks : array[0..34] of integer;
3      i,tot,max : integer;
4      avg : real;
5  begin
6      for i := 0 to 34 do
7      begin
8          writeln('Enter Marks');
9          read(marks[i]);(* Read Marks to array *)
10         tot := tot + marks[i];(* Add marks *)
11     end;
12     avg := tot/35;
13     max := marks[0];
14     for i := 1 to 34 do
15         if marks[i] > max then max := marks[i];
16     writeln('Maximum marks = ', max);
17     writeln('Average marks = ',avg);
18 end.
```

The first loop reads the 35 marks into the array and adds them to tot.

max := marks[0]; then for every other element, if marks[i] > max, max takes its value. At the end max is the highest mark.

Comments are written between (* and *).

DO IT - PRACTICAL 5 (TEXTBOOK PAGES 34 - 38)

Nested structures and arrays in Free Pascal or Lazarus. **Ask your teacher first.**

1	Type **orderNos**. Run it and type A. Run it again and type D. Then try a small a: what happens?
2	Type **rep_sel**. Enter ten numbers. Count the odd and even numbers yourself and check the program.
3	Type **ictMarks** for 5 students: [0..4], for i := 0 to 4, for i := 1 to 4 and tot/5. Enter five marks; check the maximum and the average with a calculator.
4	Add tot := 0; before the first loop: give every counter its starting value.

A Fill the array

7 marks

Run the statements. Write the value of each element, then answer.

```
1  var marks : array[1..5] of integer;
2  marks[1] := 20;
3  marks[3] := 15; marks[5] := 40;
4  marks[2] := marks[3] * 2;
5  marks[4] := marks[5] + marks[3];
```

marks[1]	marks[2]	marks[3]	marks[4]	marks[5]
❶	❷	❸	❹	❺

6. How many elements does the array marks have?

7. What is the index of the first element?

B What does orderNos print?

6 marks

Write the whole output of orderNos (1.8.1) for each choice.

1. The user types **D**

OUTPUT

2. The user types **A**

OUTPUT

C What does it print?

6 marks

The array num of 1.9.4 holds 45, 75, 36, 81, 60. Write the output.

```
writeln(num[2]);
```

OUTPUT

```
writeln(num[0] + num[4]);
```

OUTPUT

```
writeln(num[3] - num[1]);
```

OUTPUT

```
for x := 1 to 3 do
    writeln(num[x]);
```

OUTPUT

```
for x := 4 downto 2 do
    writeln(num[x]);
```

OUTPUT

```
num[2] := num[2] * 2;
writeln(num[2]);
```

OUTPUT

D Complete the program

5 marks

It reads ten marks and prints the lowest. Fill the gaps.

```
1  program lowMarks(input,output);
2  var marks : array[ ❶ ] of integer;
3      i, min : integer;
4  begin
5      for i := ❷ to 9 do
6          read(marks[i]);
7      min := marks[ ❸ ];
8      for i := 1 to 9 do
9          if marks[i] ❹ min then min := marks[i];
10     writeln('Lowest mark = ', ❺ );
11 end.
```

1. .....
2. .....
3. .....
4. .....
5. .....

"Trace the program with a pencil before you run it."

- THE KING OF ICT

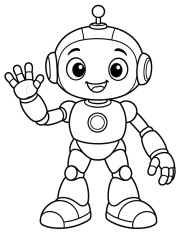
CHAPTER 6

Sub-programs and Programming Languages

HELLO MR. COMPUTER?

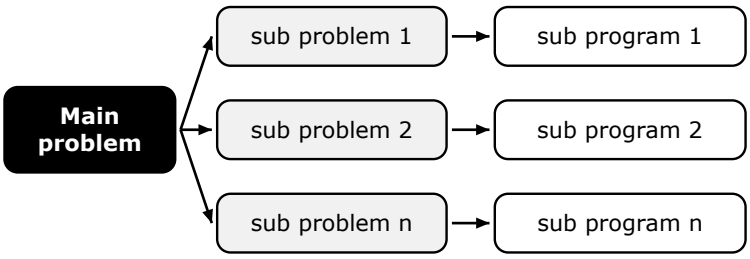
"Hello Mr. Computer?" "1010101010101010!"

The computer understands only machine language. Programs are written in **programming languages** and big programs are divided into **sub-programs**.



1.10 Use of sub-programs

When the number of sub processes increases, a program becomes complex: difficult to read, understand and maintain. So we use **sub-programs** as much as possible: the main problem is divided into sub problems, and each is solved by a sub program.



1.10.1 Types of sub-programs. A sub-program which **returns an output** back to the main program is a **function**; a sub-program which **does not return an output** is a **procedure**.

1.10.2 Introducing sub-programs. Functions and procedures are declared **before** the main program starts; the main program calls them.

```
PROGRAM  name;                          ← The heading of the program
CONST ...   VAR ...                     ← The constants and the global variables
procedure ... / function ...            ← Sub-programs: declared BEFORE the main program
BEGIN                                   ← The main program: it calls the procedures and
END.                                      the functions by their names
```

```
Procedure Name_of_Procedure( name_of_variable : data type);
Function Name_of_Function(name_of_variable : data type) : data type ;
```

WRITING SPACE 17 - Sub-programs

write what your teacher tells you

"Divide a big program into small sub-programs. Divide a big goal into small steps."

- THE KING OF ICT

E.g. - The area and the circumference of a circle: 1. with procedures, 2. with functions.

```
program procedure_circle(input,output);
const pie = 22/7;
var radius:real;

procedure getData(var radius: real);
begin
    writeln('Enter Radius');
    read(radius);
end;
procedure processArea(var radius:real);
var area:real;
begin
    area := pie * radius * radius;
    writeln('Area = ',area);
end;
procedure processCircumference(var radius:real);
var circum:real;
begin
    circum := 2 * pie * radius;
    writeln('Circumference = ',circum);
end;
begin
    getData(radius);
    processCircumference(radius);
    processArea(radius);
end.
```

1. A program built with procedures

```
program function_circle(input,output);
const pi = 22/7;
var radius:real;

function processArea(var radius:real):real;
var area:real;
begin
    area := pi * radius * radius;
    processArea := area;
end;
function processCircumference(var radius:real):real;
var circum:real;
begin
    circum := 2 * pi * radius;
    processCircumference := circum;
end;
begin
    writeln('Enter Radius');
    read(radius);
    writeln('Circumference = ',
            processCircumference(radius));
    writeln('Area = ', processArea(radius));
end.
```

2. A program with functions

1.11 Evolution of programming languages

1.11.1 A **program** is a sequence of instructions which performs a certain task using the computer; a computer language is needed to provide the instructions. **1.11.2 Low-level programming languages:**

MACHINE LANGUAGE

Understood directly by the computer: binary 0s and 1s (bits).

- Executed directly on the machine
- Fast in operation
- No need of language translating programs
- Depends on the machine: a program for one computer may not run on another
- Difficult for humans to understand (0s and 1s)

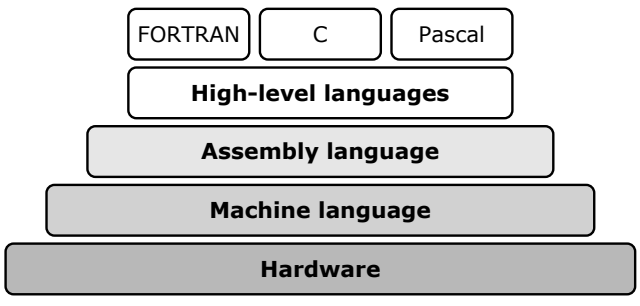
ASSEMBLY LANGUAGE

Uses simple symbols instead of 0 and 1.

- Slower than machine language
- Translated by a language translator called an assembler
- Depends on the machine
- Simple symbols make it easier to understand

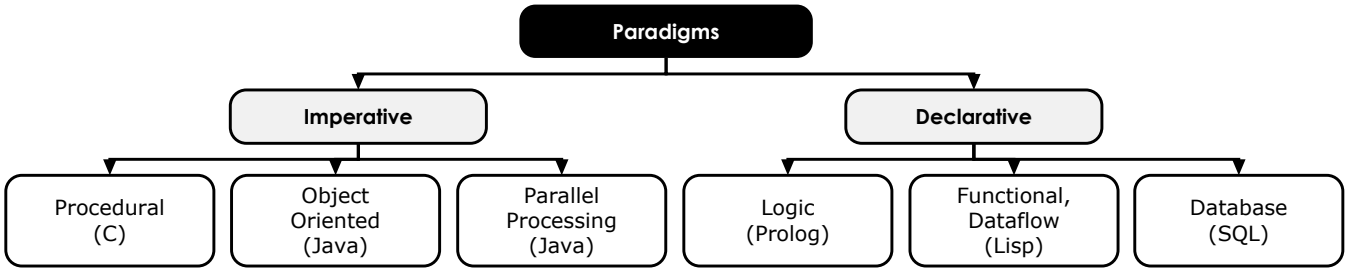
1.11.3 High-level programming languages are designed with simple English words, so the programmer understands them easily. Examples: FORTRAN, BASIC, COBOL, PASCAL, C.

- Easy to understand (simple English words)
- Translated to machine instructions before it is executed
- Does not depend on the machine



1.11.4 Programming language types

There are many different approaches to computer programming, called **programming paradigms**. Most languages come under one paradigm, but some show elements of different paradigms.



Paradigm	Languages
Imperative / Algorithmic	Algol, Cobol, PL/1, Ada, C, Modula - 3
Declarative: functional programming	Lisp, Haskell, ML, Miranda, APL
Declarative: logic programming	Prolog
Object-oriented	Smalltalk, Simula, C++, Java

Procedural and declarative paradigms

Procedural paradigm: saying HOW you achieve it	Declarative paradigm: saying WHAT you want
A well structured set of steps and procedures with problem-solving statements. Pascal has procedural features. **E.g.** 1. Keep block A. 2. Keep block B on top of block A. 3. Keep block C on top of block B.	Shows the calculations and / or logic without a control flow: what you want, not how to solve it. The computer finds the solution. Related to Artificial Intelligence. **E.g.** A pillar which consists of 3 blocks.

Structured and object-oriented paradigms

A **structured** program divides the system into sub-systems with a top-down flow (library: record loans, add resources, report fines); Pascal is structured. **Object-oriented programming** is based on **objects**: data (**properties**, called attributes) and **methods** (code that handles the data). A **class** creates objects of the same type (library: catalog, book, librarian, library).

	Class: Dog	Object: Tommy (an instance)
Property (data)	Color, Eye Color, Height, Length, Weight	Color - Gray, White and Black; Eye Color - Blue and Brown; Height - 18 Inches; Length - 36 Inches; Weight - 30 Pounds
Methods (code)	Sitting, Lying, Shaking, Walking	Sitting, Lying, Shaking, Walking

Programming and scripting. Programming languages have hard and fast syntax rules and need to be **compiled**; scripting languages are **interpreted**. JavaScript and PHP are scripting languages.

WRITING SPACE 18 - Low-level and high-level languages

1.11.5 Language translators

A program in any language except machine language (object code) must be translated to machine language before it is executed: assembly language by an **assembler**, a high-level language by an **interpreter** or a **compiler**.

INTERPRETER

Translates each statement of a high-level program to machine language one by one and executes it instantly. With a syntax error, the program runs till the error is reached. It translates the code **each time** the program is executed.

COMPILER

Translates the **entire** program to machine language before it is executed. With syntax errors, it is not translated and the errors are highlighted. Translated once, it can be executed any number of times.

WRITING SPACE 19 - Language translators write what your teacher tells you

DO IT - PRACTICAL 6 (PAGES 41 - 42 AND ACTIVITIES, PAGE 49)

Sub-programs in Free Pascal or Lazarus, then the two activities. **Ask your teacher first.**

1	Type **procedure_circle**. Run it with the radius 7: the area is 154 and the circumference is 44.
2	Type **function_circle** and run it with the same radius. Which lines return the outputs?
3	Delete the ; after one end. Compile: the compiler stops and highlights the error before the program runs.
4	**Activity (page 49):** consider Car as a class. Write its objects, its properties and its methods.
5	**Activity (page 49):** compare procedural and declarative, structured and object oriented, programming and scripting.

WRITING SPACE 20 - Programming paradigms write what your teacher tells you

A Procedure or function?

6 marks

Tick one box in each row.

Statement	Procedure	Function
It returns an output back to the main program.	☐	☐
It does not return an output back.	☐	☐
getData(radius); in procedure_circle calls a ...	☐	☐
processArea := area; gives back its result.	☐	☐
Its heading ends with : real;	☐	☐
procedure processArea(var radius:real);	☐	☐

B Which language level?

8 marks

Tick the language each sentence describes.

Feature	Machine	Assembly	High-level
Written using 0s and 1s (bits).	☐	☐	☐
Uses simple symbols instead of 0 and 1; translated by an assembler.	☐	☐	☐
Uses simple English words; easy to understand.	☐	☐	☐
Executed directly on the machine; no translator is needed.	☐	☐	☐
Does not depend on the machine.	☐	☐	☐
Pascal, BASIC, COBOL, FORTRAN and C.	☐	☐	☐
Fast, but difficult for humans to understand.	☐	☐	☐
Slower than machine language, but depends on the machine.	☐	☐	☐

C Assembler, interpreter or compiler?

6 marks

Tick the translator.

Statement	Assembler	Interpreter	Compiler
Translates an assembly language program to machine language.	☐	☐	☐
Translates and executes one statement at a time.	☐	☐	☐
Translates the whole program before it is executed.	☐	☐	☐
Translates the code each time the program is executed.	☐	☐	☐
Translated once, the program can be executed any number of times.	☐	☐	☐
With a syntax error, the program runs till the error is reached.	☐	☐	☐

"Every language needs a translator. Every student needs a teacher."

- THE KING OF ICT

REVISION

Unit 1 Part 2 in One Page

Read this the night before the test

Topic	The key facts
IF ... THEN	The statement is executed only if the condition is satisfied.
IF ... THEN ... ELSE	Statement 1 if the condition is satisfied, if not statement 2.
Nested IF and CASE	Many conditions one after the other: nested IF. One variable with many values: CASE ... of ... Else ... End;
FOR ... DO	Number of repetitions known: FOR X := 1 TO 5 DO (5 times), FOR X := 10 DOWNTO 5 DO (6 times).
WHILE ... DO	The condition is checked at the beginning; it runs while the condition is true.
REPEAT ... UNTIL	Runs once before the condition is checked; it stops when the condition becomes true.
Nested structures	A repetition inside a selection (orderNos); a selection inside a repetition (rep_sel).
Arrays	Data of the same type under one name in adjoining spaces: var num : array[0..4] of integer; = 5 elements.
Using arrays	num[3] is the 4th element of array[0..4]. A FOR loop reads or prints all the elements.
Sub-programs	Function: returns an output to the main program. Procedure: does not. Declared before the main program.
Languages	Machine language (0 and 1, fast, machine dependent); assembly (symbols, assembler); high-level (English words, translated).
Paradigms	Imperative (procedural, object oriented, parallel) and declarative (logic, functional, database). Procedural: how. Declarative: what.
Translators	Assembler; interpreter: one statement at a time, each run; compiler: the whole program once.
Scripting	Programming languages are compiled; scripting languages are interpreted (JavaScript, PHP).

TICK WHEN YOU CAN DO IT

- [ ] I can write IF, IF ... ELSE, nested IF and CASE statements.
- [ ] I can choose and write FOR, WHILE and REPEAT ... UNTIL loops.
- [ ] I can trace a loop and write the output of a program.
- [ ] I can declare an array and use it with a FOR loop.
- [ ] I can write procedures and functions and call them.
- [ ] I can compare machine, assembly and high-level languages.
- [ ] I can explain an assembler, an interpreter and a compiler.

WORD POWER

Unit 1 Part 2 Word List

Twenty-six words you must know

Word	What it means
IF ... THEN	Runs a statement only if a condition is satisfied.
ELSE	The part of an IF statement that runs when the condition is not satisfied.
Nested IF	An IF statement inside another IF statement.
CASE	Selects one choice by the value of one variable.
FOR ... DO	Repeats a known number of times (TO counts up, DOWNTO counts down).
WHILE ... DO	Repeats while a condition is true; checked at the beginning.
REPEAT ... UNTIL	Repeats until a condition becomes true; checked at the end.
Infinite loop	A loop whose condition never stops it.
Block	Statements between begin and end; run as one.
Nested structure	A control structure inside another control structure.
Array	Data items of the same type stored under one name.
Index	The number of an element in square brackets: num[3].
Element	One value stored in an array.
Sub-program	A part of a program that does one sub-task.
Procedure	A sub-program that does not return an output.
Function	A sub-program that returns an output to the main program.
Machine language	Instructions in 0s and 1s; run directly by the processor.
Assembly language	A low-level language of simple symbols.
High-level language	A language of simple English words: Pascal, C.
Paradigm	An approach to programming, e.g. procedural or declarative.
Object	Data (properties) and methods (code) together.
Class	The basic structure of OOP; it creates objects of the same type.
Scripting language	An interpreted language such as JavaScript or PHP.
Assembler	Translates assembly language to machine language.
Interpreter	Translates and executes one statement at a time.
Compiler	Translates the whole program to machine code before it runs.

"Know your words: a compiler, an array, a function. They are the keys of the O/L paper."

- THE KING OF ICT

"A little programming every week beats a lot of programming the night before the exam."

- THE KING OF ICT

END OF PART 2

Unit 1 Part 2 Test

Time: 45 minutes Total: 40 marks

Name _____ **Class** _____ **Marks** ____

1 Choose the correct answer

8 marks

Underline your answer.

1. Which statement is easier when one variable has many conditions?
   (a) FOR (b) CASE (c) WHILE (d) REPEAT
2. How many times does FOR X := 5 TO 10 DO repeat?
   (a) 5 (b) 6 (c) 10 (d) 15
3. A WHILE ... DO loop checks its condition
   (a) at the beginning (b) after one run (c) only when true (d) never
4. How many elements has var M : array[0..4] of integer;?
   (a) 4 (b) 5 (c) 6 (d) 0
5. A sub-program that returns an output is a
   (a) procedure (b) function (c) constant (d) loop
6. Which language is executed directly on the machine?
   (a) machine (b) assembly (c) Pascal (d) Java
7. Which of these is a scripting language?
   (a) PHP (b) Pascal (c) COBOL (d) FORTRAN
8. Which translator translates the whole program before it runs?
   (a) interpreter (b) compiler (c) assembler (d) editor

2 Fill in the blanks

8 marks

One word (or sign) in each blank.

1. When there are multiple conditions one after the other, a ..... IF is used.
2. FOR count := 10 ..... 1 do counts down from 10 to 1.
3. A REPEAT ... UNTIL loop stops when its condition becomes ..... .
4. A loop whose condition never stops it is an ..... loop.
5. In var num : array[0..4] of integer; the last element is num[.....].
6. Functions and procedures are declared ..... the main program.
7. Machine language and assembly language are ..... -level languages.
8. A ..... translates the entire program to machine language before it is executed.

3 What is the output?

8 marks

Write the output of each part of a program (2 marks each).

```
x := 7;
if x MOD 2 = 0 then
    writeln('Even')
else
    writeln('Odd');
```

OUTPUT

```
for i := 3 downto 1 do
    writeln(i * 2);
```

OUTPUT

```
k := 1;
while k < 20 do
    k := k * 3;
writeln(k);
```

OUTPUT

```
a[1] := 4; a[2] := 9;
a[3] := a[1] + a[2];
writeln(a[3]);
```

OUTPUT

---

4 Answer these

16 marks

Write in the space given.

1. Write a FOR loop that prints the numbers from 10 down to 1. (2 marks)

2. Rewrite this IF statement as a CASE statement for the marks 0..100 (0..49: grade 'S', 50..100: grade 'P'; any other mark: Invalid Marks). (4 marks)

```
If Marks <= 49 then
    Grade := 'S'
Else
    Grade := 'P';
```

3. Declare an array for 5 whole numbers and write a FOR loop that reads 5 numbers into it. (4 marks)

4. Write one difference between a procedure and a function. (2 marks)

5. Write two differences between an interpreter and a compiler. (4 marks)

> "Every loop you trace today is a mark you collect in the O/L examination."
>
> - THE KING OF ICT